Introduction

COVID-19 is the disease caused by a new coronavirus named SRARS-CoV-2 that emerged in 2019, it is extremely contagious and spreads through droplets released into the air when an infected person coughs or sneezes. Various studies have been conducted worldwide to find a testing method that can quickly and accurately detect the virus. The goal of this project is to predict confirmed COVID-19 cases based on hematology data. The data set consists of 598 samples, each has 15 types of tests. A subset of 14 tests and 598 samples will be used to train the neural network to classify patients and the remaining samples will be fed to the network to evaluate the performance.

Methodology

1.1 Data Preprocessing

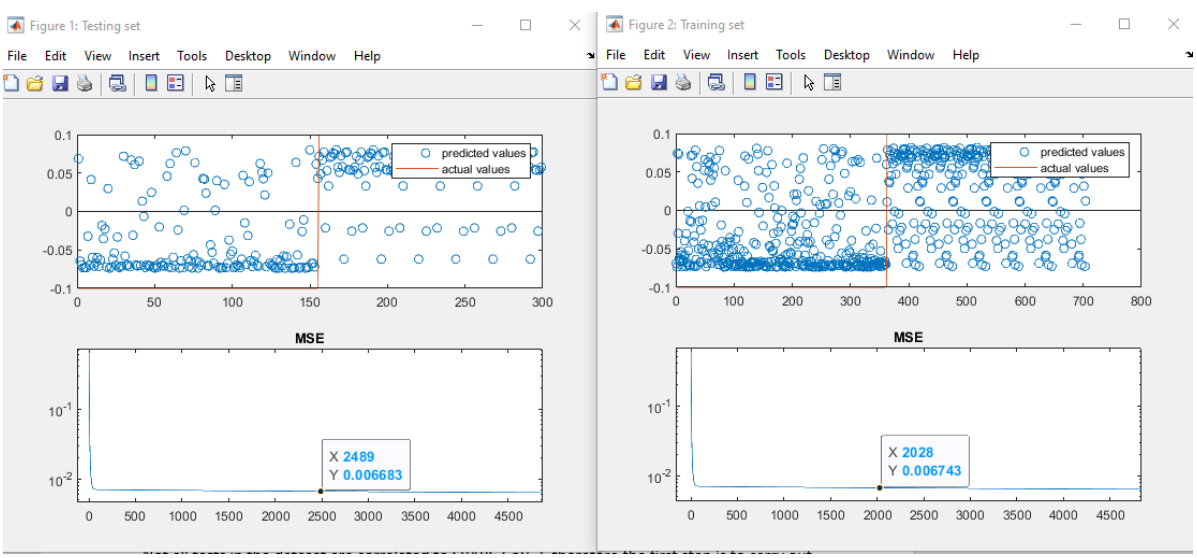
Not all tests in the dataset are correlated to SRARS-CoV-2, therefore the first step is to carry out investigations on these medical terminologies. The Leukoerythroblastic is a condition in which the leukocyte count is above the normal range, usually indicates a significant deviation to the immune system[1] Thus leukocyte will be taken as P1. Researchers from Coney Island Hospital have compared the eosinophil results of routine CBC(complete blood count) of patients and found a noticeable correlation between eosinophil counts and SARS-CoV-2.[2] Therefore eosinophils results will be noted as P2. Also, an analysis of 7612 COVID-19 patients revealed that patients with severe disease tend to have a lower platelet count, which proposes that platelet count should be used as P3.[3]This means Mean platelet volume can also be used as an input. A study has found that monocytes are key contributors to cytokine storm in COVID-19, infected patients are inclined to have higher monocyte counts, which means high values of monocyte counts are a strong indicator of Covid-19 presence. In conclusion, only leukocytes, eosinophils platelet, mean platelet volume and monocyte(5 inputs) are used to train and test the network.

1.2 Inputs Selection

Next is to decide which combinations of inputs generate the best results, a random two-hidden-layers neuron network was construed(P-4-2-1). All parameters are equal except the size of the input and first layer weight matrix. Testing metrics will be discussed in section 3.9. All values given in the table are for evaluating the testing set only.

Test 1 3 inputs(leukocytes, eosinophils, platelet)

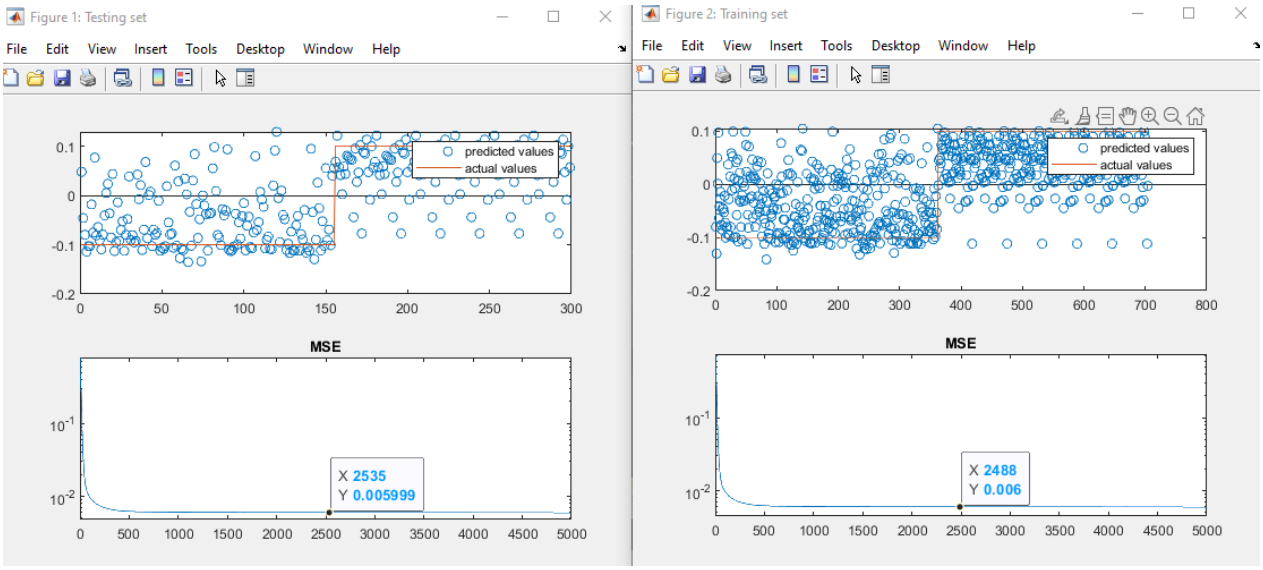
Classification plots:



Accuracy	0.8395

True positive rate	0.7857
True negative rate	0.9084
Negative predictive value	0.7677
Positive predictive value	0.9167
F1 score	0.8462
Final mean square error	0.0063

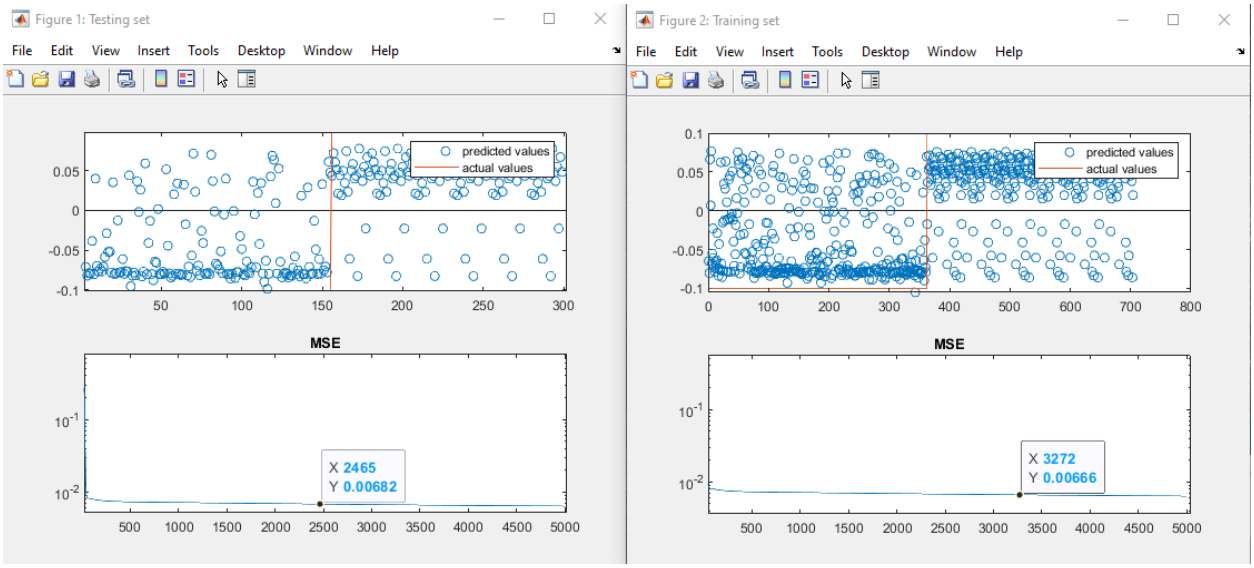
Test 2 4 inputs(leukocytes, eosinophils, platelet, mean platelet volume)



Accuracy	0.8294
True positive rate	0.7925
True negative rate	0.8714
Negative predictive value	0.7871
Positive predictive value	0.8750
F1 score	0.8317
Final mean square error	0.006

By comparing these four plots, it can be concluded that the same network performs worse when mean platelet volume is added as an input.

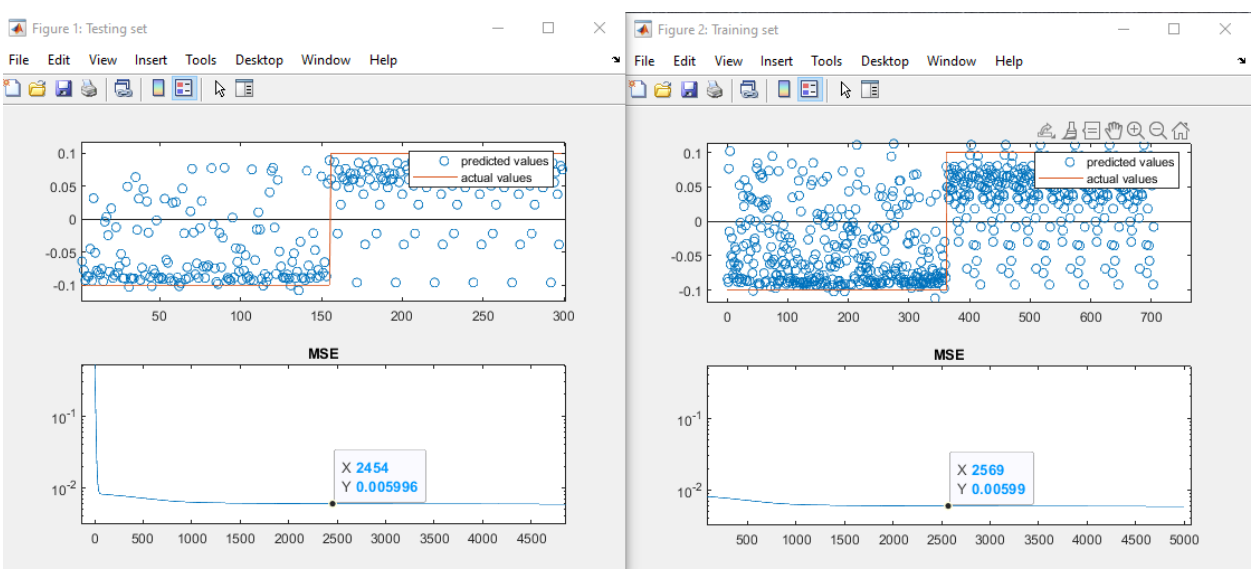
Test 3 5 inputs(leukocytes, eosinophils, platelet, mean platelet volume, monocyte)



Accuracy	0.8395
True positive rate	0.8077
True negative rate	0.8741
Negative predictive value	0.8065
Positive predictive value	0.8750
F1 score	0.84
Final mean square error	0.0064

It can be seen that this plot has a less dense line around 0.1, which means the network tends to perform worth when predicting positive cases.

Test 4 4 inputs(leukocytes, eosinophils, platelet, monocyte)



Accuracy	0.8261
True positive rate	0.7875
True negative rate	0.8705
Negative predictive value	0.7806
Positive predictive value	0.8750
F1 score	0.8289
Final mean square error	0.0059

Conclusion: All four tests have similar results except test 3, therefore the mean platelet will be removed from the input data set. However, this testing method is flawed as the initial values of the weight matrices are generated randomly. One lucky run might help the network to locate the global minima of the lost function. As a result, the configurations for test 4 will be both used to design the neural network.

1.3 Oversampling

It can be observed that the dataset is exceedingly unbalanced, with 517 negative cases and only 81 positive cases. The classifier will generate a biased learning model that has poor accuracy over the positive cases. To avoid this, there are two commonly used techniques to resample the input matrix. One is known as undersampling, an approach that involves randomly selecting samples from the majority class and deleting them. This method is only effective if the minority class has sufficient samples. Consequently, the minority class has been randomly duplicated to match the size of the majority class.

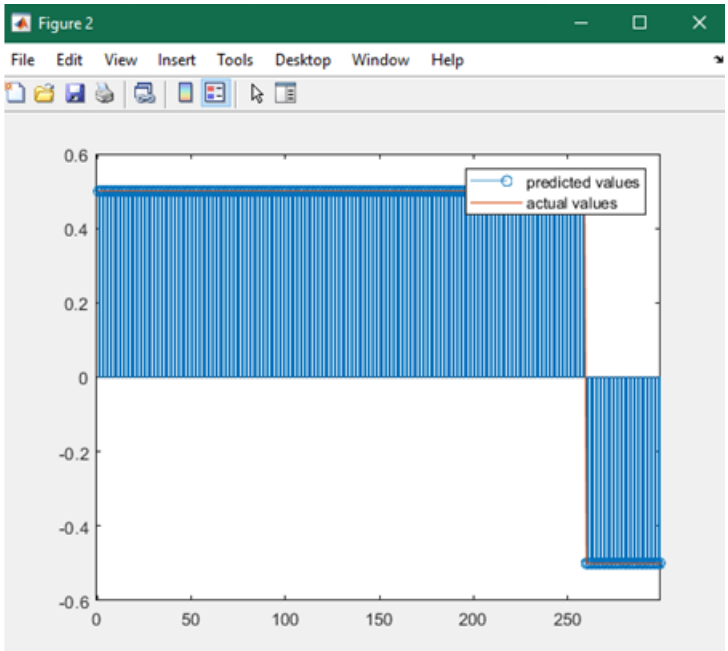
1.4 Shuffling Training Set

This method has been applied during the oversampling procedure to randomly sort the training set, it has also been employed at the end of each iteration to shuffle the training set. Since the network will be trained by the backpropagation algorithm, this steepest descent based algorithm is susceptible to stuck at local minima if the optimal solution lies nearby. Therefore the training matrix has been shuffled at each epoch.

1.5 Avoiding Overfitting

Overfitting is a modelling error that occurs when the network is closely fit to a very limited dataset, the network will produce an output far from the correct value if the input was not contained in the training set. To prevent this problem, the network should have fewer parameters compared to the number of data points.

A 3-100-1 network is given below to illustrate this problem



Training set: 99.9% accuracy

The testing set has a terrible prediction rate for both positive and negative cases.

For a one-hidden layer design(a-num_n-1), if the number of the inputs is denoted as a, then the maximum number of neurones must satisfy the following equation:

a*(max_n)+a*1+num_n+num_n+1<=numer of samples

For a two-hidden layer design(a-num_n1-num_n2-1) the maximum number of neurones must satisfy the following equation:

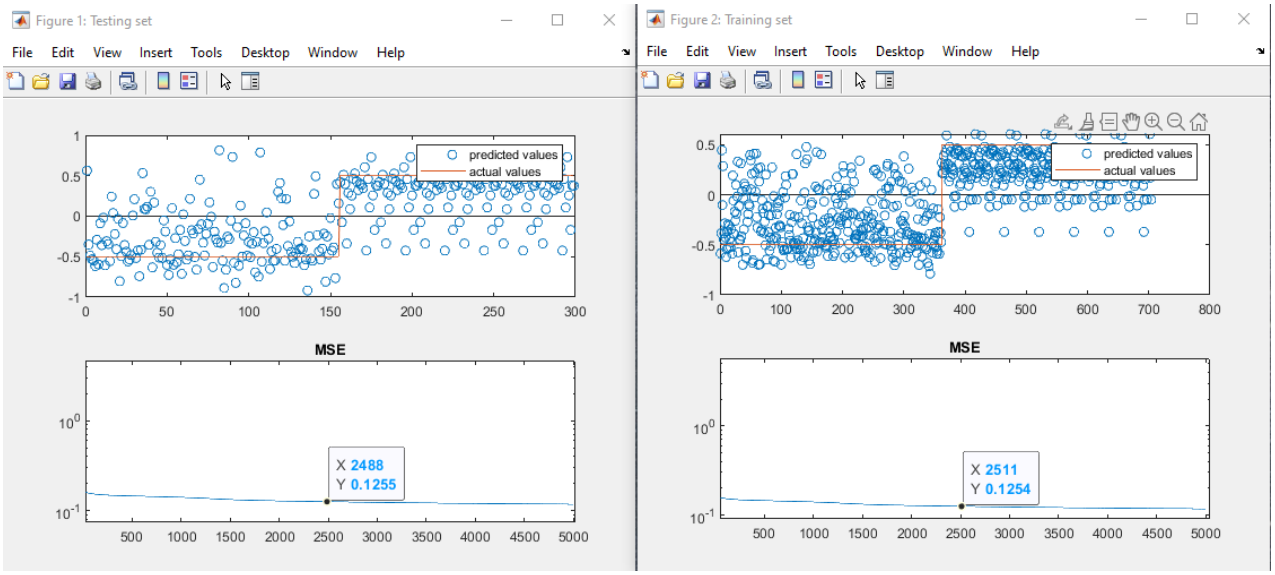
a*num_n1+num_n1*num_n2+num_n2*1+1+num_n2+num_n1<=number of samples

1.6 Neuron Network Design

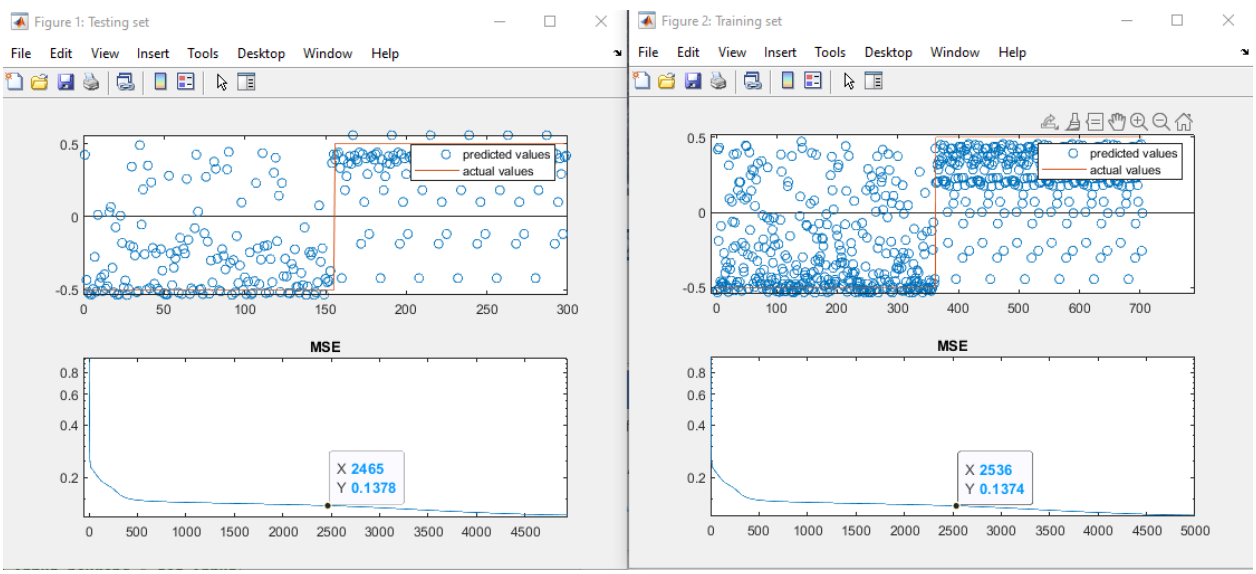
1.6.1 Number of Layers

The number of hidden layers should be less than two[5], and two hidden layers usually better than one.[6]

Test 1 one hidden layer,4 inputs (4-10-1)



Test 2 two hidden layers,4 inputs(4-4-2-1)



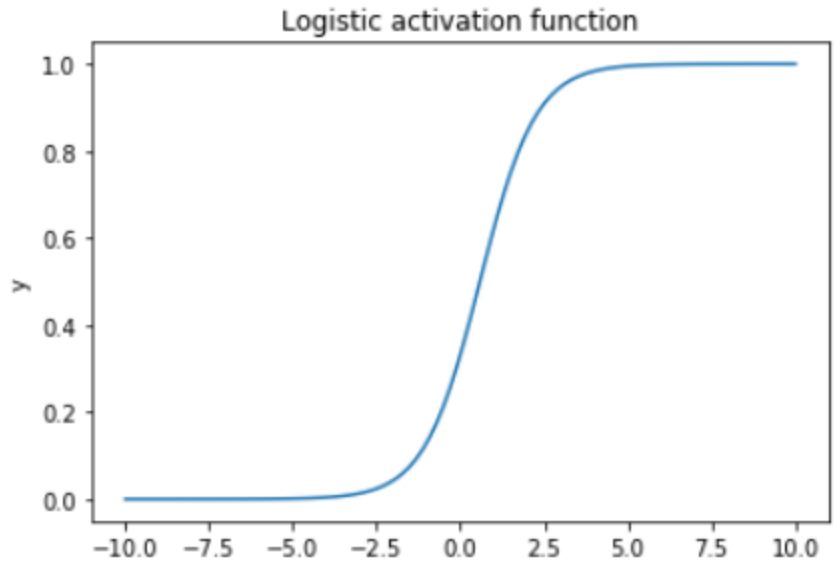
Two hidden layers network has noticeable better performance, therefore the number of layers will be two.

1.6.2 Number of Neurons

The number of neurons is decided through trial and error and is shall not pass the maximum value calculated from the equations listed in section 3.5 to avoid overfitting. The result will be shown in section 1.7.

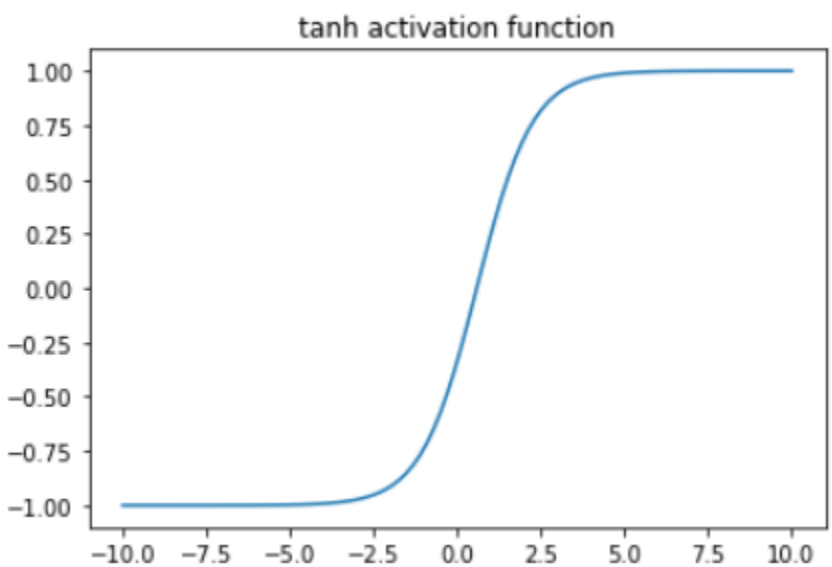
1.6.3 Activation function

1.6.3.1. Logsig



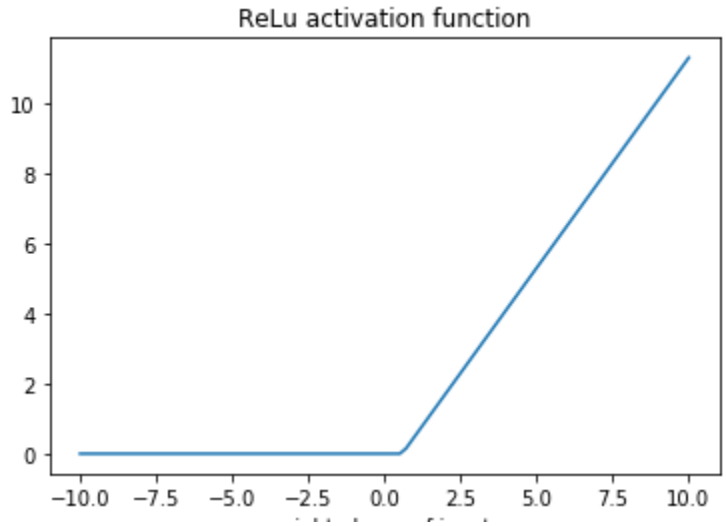
Logisig transfer function is used if the target values are zero and a positive value(ie, 0.5 for positive cases and 0 for negative cases)

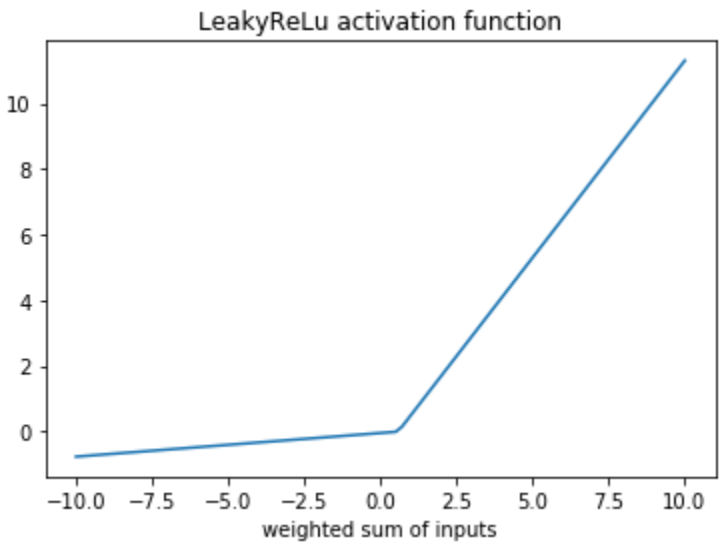
1.6.3.2 Tansig



Tansig transfer function is used if the target values are zero and a positive value(ie, 0.5 for positive cases and -0.5 for negative cases)

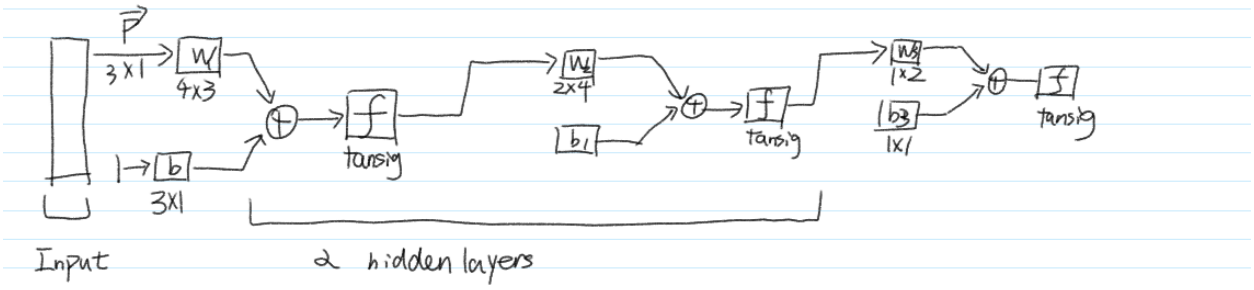
1.6.3.3 Relu and Leaky Relu





A neuron is said to be saturated if it reaches the maximum or negative values, the derivative of this peak value is zero, meaning there is no updating to the weight. This is common for logistic functions due to the flat curves at two sides and this issue can be avoided using Rectified linear unit transfer functions. However, through trial and error, networks with Relu layers have similar performance compared to all tansig layers network.

1.7 Neuron Network Diagram



1.8 Backpropagation Algorithm

$$\hat{F} = \vec{e}^T_k \cdot \vec{e}_k = (\vec{t}-\vec{a})^T(\vec{t}-\vec{a}) \qquad \vec{e} = \begin{bmatrix} e_1 \\ e_2 \\ \vdots \\ e_S \end{bmatrix}$$

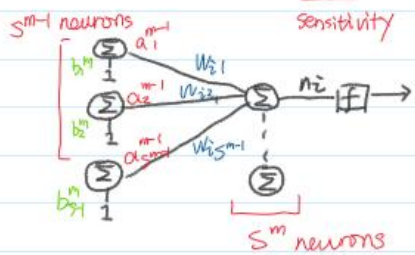
apply steepest descent to $\hat{F}$

$$\Delta W^m_{ij} = -\alpha \frac{\partial \hat{F}}{\partial W^m_{ij}}$$

$$W^m_{ij}(k+1) = W^m_{ij}(k) - \alpha \frac{\partial \hat{F}}{\partial W^m_{ij}(k)}$$

neuron # input # iteration #

$$\boxed{\frac{\partial \hat{F}}{\partial W_{ij}^m} = \underbrace{\frac{\partial \hat{F}}{\partial n_i^m}}_{\text{sensitivity}} \frac{\partial n_i^m}{\partial W_{ij}^m}} \quad , \quad \boxed{\frac{\partial \hat{F}}{\partial b_i^m} = \frac{\partial \hat{F}}{\partial n_i^m} \times \frac{\partial n_i^m}{\partial b_i^m}}$$



$$n_i^m = \sum_{j=1}^{S^{m-1}} W_{ij}^m a_j^{m-1} + b_i^m \quad \Rightarrow \boxed{\frac{\partial n_i^m}{\partial W_{ij}^m} = a_j^{m-1}} \; , \; \boxed{\frac{\partial n_i^m}{\partial b_i^m} = 1}$$

define $s_i^m = \dfrac{\partial \hat{F}}{\partial n_i^m}$

$$\Rightarrow \boxed{\frac{\partial \hat{F}}{\partial W_{ij}^m} = s_i^m a_j^{m-1}} \; , \; \boxed{\frac{\partial \hat{F}}{\partial b_i^m} = s_i^m}$$

$$\boxed{W_{ij}^m(k+1) = W_{ij}^m(k) - \alpha s_i^m a_j^{m-1}}$$

$$\boxed{b_i^m(k+1) = b_i^m(k) - \alpha s_i^m}$$

in matrix form:

$$\boxed{\begin{aligned} \vec{W}_{k+1} &= \vec{W}_k - \alpha \vec{s}^m (\vec{a}^{m-1})^T \\ \vec{b}_{k+1} &= \vec{b}_k - \alpha \vec{s}^m \end{aligned}} \quad \text{where } \vec{s}_m = \begin{bmatrix} \frac{\partial \hat{F}}{\partial n_1^m} \\ \frac{\partial \hat{F}}{\partial n_2^m} \\ \vdots \\ \frac{\partial \hat{F}}{\partial n_{S^m}^m} \end{bmatrix}$$

in matrix form:

$$\boxed{\begin{aligned}\vec{W}_{k+1} &= \vec{W}_k - \alpha\, \vec{s}^{\,m} (\vec{a}^{\,m-1})^T \\ \vec{b}_{k+1} &= \vec{b}_k - \alpha\, \vec{s}^{\,m}\end{aligned}}$$

where $\vec{s}_m = \begin{bmatrix} \frac{\partial \hat{F}}{\partial n_1^m} \\ \frac{\partial \hat{F}}{\partial n_2^m} \\ \vdots \\ \frac{\partial \hat{F}}{\partial n_{S^m}^m} \end{bmatrix}$

① To obtain $\vec{s}^{\,m}$, first we need to calculate sensitivity at the output layer $\frac{\partial \hat{F}}{\partial \vec{n}_i^m} = \vec{s}^{\,m} = \begin{bmatrix} \frac{\partial \hat{F}}{\partial n_1^M} \\ \frac{\partial \hat{F}}{\partial n_2^M} \\ \vdots \\ \frac{\partial \hat{F}}{\partial n_{S^M}^M} \end{bmatrix}$

$$s_i^M = \frac{\partial \hat{F}}{\partial n_i^M} = \frac{\partial (\vec{t}-\vec{a})^T(\vec{t}-\vec{a})}{\partial n_i^M} = \frac{\partial \sum_{j=1}^{S^M} (t_j - a_j)^2}{\partial n_i^M} = -2(t_i - a_i)\frac{\partial a_i}{\partial n_i^M}$$

$$\frac{\partial a_i}{\partial n_i^M} = \frac{\partial a_i^M}{\partial n_i^M} = \frac{\partial f^M(n_i^M)}{\partial n_i^M} = \dot{f}^M(n_i^M)$$

$$\Rightarrow s_i^M = -2(t_i - a_i)\,\dot{f}^M(n_i^M)$$

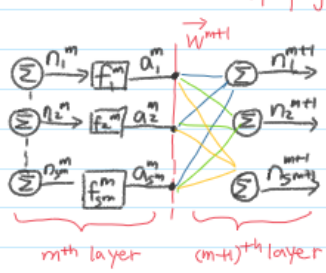
$$\vec{s}_i^M = \frac{\partial \hat{F}}{\partial \vec{n}_i^M} = \begin{bmatrix} \frac{\partial \hat{F}}{\partial n_1^M} \\ \frac{\partial \hat{F}}{\partial n_2^M} \\ \vdots \\ \frac{\partial \hat{F}}{\partial n_{S^M}^M} \end{bmatrix} = \begin{bmatrix} -2(t_1-a_1)\dot{f}^M(n_1^M) \\ -2(t_2-a_2)\dot{f}^M(n_2^M) \\ \vdots \\ -2(t_{S^M}-a_{S^M})\dot{f}^M(n_{S^M}^M) \end{bmatrix} = -2 \begin{bmatrix} \dot{f}^M(n_1^M) & 0 & \cdots & 0 \\ 0 & \dot{f}^M(n_2^M) & \cdots & 0 \\ 0 & 0 & \cdots & 0 \\ 0 & 0 & \cdots & \dot{f}^M(n_{S^M}^M) \end{bmatrix} = -2\,\dot{\vec{F}}^M(\vec{n}^M)(\vec{t}-\vec{a})$$

$\Rightarrow$ sensitivity of the output layer

$$\boxed{\vec{s}^{\,M} = -2\,\dot{\vec{F}}^M(\vec{n}^M)(\vec{t}-\vec{a})}$$

② next is to calculate sensitivities of hidden layers

$$\frac{\partial \hat{\vec{F}}}{\partial \vec{n}^m} = \left(\frac{\partial \vec{n}^{m+1}}{\partial \vec{n}^m}\right) \underbrace{\frac{\partial \hat{\vec{F}}}{\partial \vec{n}^{m+1}}}_{\text{backpropagate from the } (m+1) \text{ layer}}$$



$$\Rightarrow \frac{\partial \vec{n}^{m+1}}{\partial \vec{n}^m} = \begin{bmatrix} \frac{\partial n_1^{m+1}}{\partial n_1^m} & \frac{\partial n_1^{m+1}}{\partial n_2^m} & \cdots & \frac{\partial n_1^{m+1}}{\partial n_{S^m}^m} \\ \frac{\partial n_2^{m+1}}{\partial n_1^m} & \frac{\partial n_2^{m+1}}{\partial n_2^m} & \cdots & \frac{\partial n_2^{m+1}}{\partial n_{S^m}^m} \\ \vdots & \vdots & & \vdots \\ \frac{\partial n_{S^{m+1}}^{m+1}}{\partial n_1^m} & \frac{\partial n_{S^{m+1}}^{m+1}}{\partial n_2^m} & \cdots & \frac{\partial n_{S^{m+1}}^{m+1}}{\partial n_{S^m}^m} \end{bmatrix}$$

$$\frac{\partial n_i^{m+1}}{\partial n_j^m} = \frac{\partial \left(\sum_{l=1}^{S^m} w_{il}^{m+1} a_l^m + b_i^{m+1} \right)}{\partial n_j^m} = w_{ij}^{m+1} \frac{\partial a_j^m}{\partial n_j^m} = w_{ij}^{m+1} \frac{\partial (f(n_j^m))}{\partial n_j^m} = w_{ij}^{m+1}\, \dot{f}(n_j^m)$$

$$\frac{\partial \vec{n}^{m+1}}{\partial \vec{n}^m} = \vec{W}^{m+1}\, \dot{\vec{F}}(\vec{n}^m) \quad , \text{where } \dot{\vec{F}}(\vec{n}^m) = \begin{bmatrix} \dot{f}^m(n_1^m) & 0 & \cdots & 0 \\ 0 & \dot{f}^m(n_2^m) & \cdots & 0 \\ \vdots & \vdots & & 0 \\ 0 & 0 & \cdots & \dot{f}^m(n_{S^m}^m) \end{bmatrix}$$

$$\vec{s}^{\,m} = \frac{\partial \hat{\vec{F}}}{\partial \vec{n}^m} = \left(\frac{\partial \vec{n}^{m+1}}{\partial \vec{n}^m}\right)^T \frac{\partial \hat{\vec{F}}}{\partial \vec{n}^{m+1}} = \dot{\vec{F}}^m(\vec{n}^m)(\vec{W}^{m+1})^T \frac{\partial \hat{\vec{F}}}{\partial \vec{n}^{m+1}}$$

$$\Rightarrow \boxed{\vec{s}^{\,m} = \dot{\vec{F}}(\vec{n}^m)\,(\vec{W}^{m+1})^T \vec{s}^{\,m+1}}$$

Somary

Step ①

$$\vec{a}^0 = \vec{p}$$

$$\vec{a}^{m+1} = f^{m+1}(W^{m+1}a^m + b^{m+1}) \quad m = 0, 1, \cdots M-1$$

$$\vec{a} = \vec{a}^M$$

last layer $\vec{s}^M = -2\dot{\vec{F}}^M(\vec{n}^M)(\vec{t} - \vec{a})$

$$\vec{s}^m = \dot{\vec{F}}^m(\vec{n}^m)(\vec{W}^{m+1})^T s^{m+1} \quad m = M-1, 1, 2 \cdots$$

update:

$$\vec{W}^m(k+1) = \vec{W}(k) - \alpha \vec{s}^m (\vec{a}^{m-1})^T$$

$$\vec{b}^m(k+1) = \vec{b}^m(k) - \alpha \vec{s}^m$$

1.9 Training Testing scheme

70 percent of data were used for training and the remaining are used for testing. Therefore, there are 362/517 negative cases and 57/81 positive cases in the training set. After oversampling, there are 342 positive cases and 362 positive cases in the training set, 155 positive cases and 144 negative cases in the testing set.

The following metrics will be used to evaluate the performance of the network[7]

Sensitivity, recall or true positive rate(TPR)= Ture positive/(True positive+False negative)

Selectivity or true negative rate(TNR)=TN/(TN+FP)

Positive Predictive Value(PPV)= TP/(TP+FP)

Negative Predictive Value (NPV) = TN/(TN+FN)

F1 score =2*(PPV*TPR)/(PPV+TPR)
Mean Square error

The target value is set to be 0.1 and -0.1. This is because the output layer is tansig, which can only generate a value between [0 1], target values were randomly chosen in this region.

From zero to the centre of the plot, if a point is located below 0, then it is defined as true negative and vice versa. If a point is located above 0, then it is defined as true positive and vice versa.

Results and Discussion

2.1

Learning Rate=0.0001

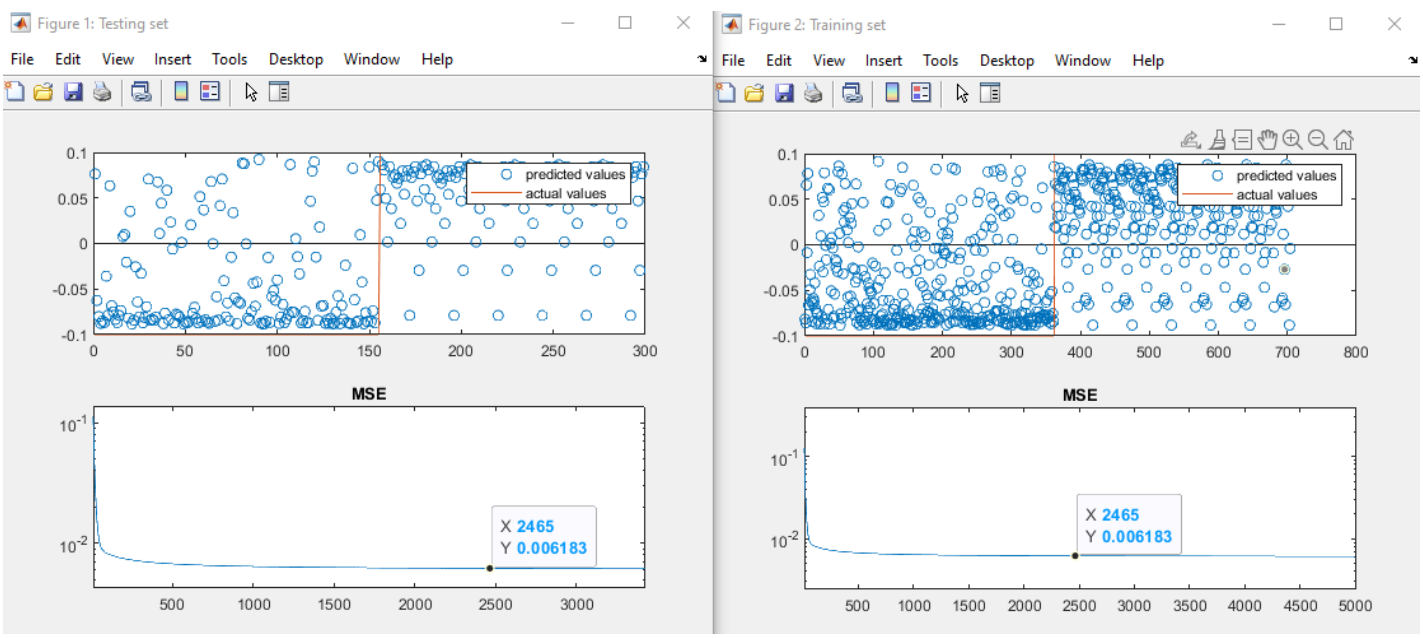
Iterations = 5000

Training error curve will be presented in section 4.3

2.2

At first, the weight matrices were initialized to a random value between 1 and 0, then the program will propagate the input forward through the network and pass the sensitivities backward from the output layer. Finally, the weight and biases were updated using the rules derived in section 3.8. At the end of each epoch, the training set was shuffled due to the reason introduced in section 3.4. Instead of using a threshold value for MSE to break the while loop, the number of the epoch was predefined before the while statement to quit the loop.

2.3 & 2.4



In the first region if the point is above zero, then it is misclassified. In the second region if the point is below zero, then it is misclassified. If a point is closer to 0.1 or -0.1, then that prediction is said to be a good prediction.

Testing set

Accuracy	0.8562
True positive rate	0.8098
True negative rate	0.9188
Negative predictive value	0.8
Positive predictive value	0.9167
F1 score	0.8599
Final mean square error	0.006

Training set

Accuracy	0.6051
True positive rate	0.6328
True negative rate	0.5964
Negative predictive value	0.7548
Positive predictive value	0.3778
F1 score	0.8489

2.5

The training set table shows that this network design has a very high positive predictive value. The network was designed to give priority to PPV and TPR, this is because a false positive might cause more damage to society. In other words, if an infected person was falsely classified as a negative case, he or she will not be under self-quarantine and could spread the virus to others.

For the testing set, the network is very effective, but it performs much worse for training data. This is mostly due to oversampling, the positive cases have been repeated 6 times, which could cause the network to favour positive cases. In the testing set plot, there are two misclassified points in the second region repeating 6 times, as the total number of samples is smaller, this issue has little effect on the testing set. On the contrary, there are 704 samples in the training set and 9 misclassified positive in the second region, so the total number of misclassified positive points is 9*6=54, which will have a significant impact on TPR TNR PPV and accuracy as these values are a function of false positive.

Another reason could just be that the training set has 7 times more data compared to the testing set. More data leads to lower accuracy.

The most important reason is that the dataset is not sufficient to train any network, with only 81 positive cases, no network can precisely model the function between inputs and outputs.

The network has consistent performance for both data set, so overfitting did not happen. Saturation did not happen because no point on the graph is larger than 0.1 or smaller than -0.1.

Conclusion

The purpose of this project is to classify COVID-19 patients as positive or negative cases. First, the data set was divided into two sets, one with 70% of the data set and the other 30% was used to evaluate the performance of the network. To prevent the issue resulted from the backpropagation algorithm being stuck at the local minima, the training set was randomly shuffled at the end of each iteration. The network is very effective in predicting the testing data, and relatively ineffective in predicting the training data. This is expected as no study has confirmed that blood test results can be widely used as COVID-19 test.

References

[1] P. A. and M. UC Davis Health, “What the blood tests of a COVID-19 patient can tell us,” *UC Davis Health*, 32-Mar-2021. [Online]. Available: https://health.ucdavis.edu/health-news/newsroom/what-the-blood-tests-of-a-covid-19-patient-can-tell-us/2020/03. [Accessed: 16-Dec-2020].

[2] E. Henderson, “Eosinophil count can aid in early detection of COVID-19 in patients,” *News Medical Life Sciences*, 16-Jul-2020. [Online]. Available: https://www.news-medical.net/news/20200716/Eosinophil-count-can-aid-in-early-detection-of-COVID-19-in-patients.aspx. [Accessed: 16-Dec-2020].

[3]G. D. Wool and J. L. Miller, “The Impact of COVID-19 Disease on Platelets and Coagulation,” Pathobiology, 13-Oct-2020. [Online]. Available: https://www.karger.com/Article/FullText/512007. [Accessed: 17-Dec-2020].

[4] B. D. Pence, “Severe COVID-19 and ageing: are monocytes the key?,” GeroScience, 2020. [Online]. Available: https://www.ncbi.nlm.nih.gov/pmc/articles/PMC7299454/. [Accessed: 17-Dec-2020].

[5] R. Lippmann, "An introduction to computing with neural nets," in IEEE ASSP Magazine, vol. 4, no. 2, pp. 4-22, Apr 1987, DOI: 10.1109/MASSP.1987.1165576.

[6] A.J Thomas, Miltos Petridis, S.D. Walters, S. M. Gheytassi, and Robert E Morgan, “Two Hidden Layers are Usually Better than One” School of Computing Engineering and Mathematics, University of Brighton, United Kingdom

[7]S. B. Vardeman, *Engineering statistics*, 2nd ed., vol. 1, 1 vols. Dortmund, United States of America: Univ., SFB 475, 2003.